

ИИ / CI/CD / ЭКСПЛУАТАЦИЯ

Технический долг и архитектурный долг: не путайте их

Архитектурный долг незаметно саботирует инициативы в области искусственного интеллекта, облачных технологий и трансформации. Узнайте, в чем разница, как диагностировать эти системные проблемы и устранить их.

22 октября 2025 года, 6:00 [Надежда Столбовская](#)



Изображение предоставлено Висенте Каргесом с сайта Shutterstock.

Разработчики, менеджеры проектов и даже руководители используют термин «технический долг» для объяснения задержек, нестабильности или растущих затрат на обслуживание. Его значение простое и понятное: мы идем на компромиссы, чтобы выпустить продукт быстрее, берем на себя «долг», а потом погашаем его, исправляя ошибки, рефактора или переписывая код.

Однако существует еще один, гораздо более опасный вид долга — архитектурный долг. В отличие от технического долга, архитектурный долг не цен в запросе на слияние. Он не проявляется в виде неработающего дублированного теста или уязвимости в системе безопасности, обнаруженной

проблема, и она часто дает о себе знать только тогда, когда программа трансформации буксует, **миграция в облако не удастся** или инициатива по внедрению искусственного интеллекта не масштабируется.

Так почему же компании так часто путают **технический долг** с архитектурным? И почему эта путаница обходится бизнесу в миллионы?

Метафора «Дом против города»

Представьте себе дом. Если сломается лестница, потечет труба или выйдет из строя электропроводка, это сразу заметят все. Это видимые проблемы, которые требуют немедленного ремонта.

С точки зрения информационных технологий, это технический долг: локальная проблема в кодовой базе, тестовом покрытии или инфраструктуре. Обычно с ней может справиться одна команда или даже один инженер. Это может быть болезненно, но проблема ощутима, диагностируема и зачастую хорошо изучена.



🔍 Масштабирование

А теперь представьте себе целый город. Каждый дом может быть свежеевыкрашенным, каждая квартира — отремонтированной, каждая комната — в идеальном состоянии. Но если дорожная сеть города плохо спроектирована, водоснабжение разрозненное, а правила зонирования ослепительны, город постепенно придет в упадок. Пробки парализуют

Это архитектурный долг. Он проявляется не в виде одной сломанной ступеньки, а в системных сбоях, вызванных несогласованностью и отсутствием координации во всей системе.



🔍 Масштабирование

ТРЕНДОВЫЕ ИСТОРИИ

1. [Groundcover eyes visibility gap in agentic AI monitoring by targeting multi-step workflows](#)
2. [SRE Housekeeping: Where to Get Started for New Hires](#)
3. [Andrej Karpathy's 630-line Python script ran 50 experiments overnight without any human input](#)
4. [Technical Debt vs. Architecture Debt: Don't Confuse Them](#)
5. [Kubernetes co-founder Brendan Burns: AI-generated code will become as invisible as assembly](#)

Тот же принцип применим и к ИТ-инфраструктуре. Команда разработчиков может создавать чистый, модульный код. Конвейеры CI/CD могут работать безупречно, а все тесты — проходить успешно. Однако за этим фасадом часто скрываются несколько дублирующих друг друга платформ с хрупкими,

The result is predictable: Every new initiative, from digital transformation to AI adoption, encounters hidden friction and costly delays.

The metaphor matters because it highlights visibility versus invisibility. Technical debt is obvious — like a leaky roof you can't ignore. Architecture debt is subtle — like poor city planning that only becomes undeniable when gridlock makes life unworkable. And while fixing a broken stair is straightforward, redesigning an entire transportation system takes years, coordination and investment.

Why Companies Confuse Technical and Architecture Debt

- **The symptoms look deceptively similar.** Both types of debt produce the same visible symptoms: delays, outages and higher costs. But the causes differ: Technical debt stems from code shortcuts, while architecture debt arises from systemic flaws such as platform duplication or broken data governance. Because the outcomes look alike, organizations often mislabel everything as “tech debt,” missing the deeper structural cause.
- **The focus stays on fixing code.** Executives often fall back on a default response: “If we just hire more developers, we'll clear the backlog.” This works when the problem is technical debt — adding engineers can indeed accelerate refactoring or bug fixes. But architecture debt doesn't yield to brute force. Ten extra developers cannot fix platform sprawl, reconcile fragmented data models or redesign an integration landscape. These are structural issues that require governance, architecture boards and cross-domain design, not simply more coding power.
- **Architecture debt hides in plain sight.** Technical debt has clear indicators: bug density, test coverage, code complexity scores. These appear naturally in developer dashboards and project reports. Architecture debt, however, rarely shows up in Jira tickets or automated scans. Few organizations track duplicate platforms, undocumented interfaces or integration complexity. Without such metrics, architecture debt remains invisible until a major transformation grinds to a halt.
- **Organizational silos reinforce the confusion.** Most teams optimize for their own scope. A development squad ensures its microservice runs smoothly. A project manager hits local milestones. A vendor delivers a contract. But no one owns the end-to-end structure — the *city* rather than the *house*. Architecture decisions become scattered: One domain picks a new Software as a Service tool, another adopts its own data model, a third chooses a different cloud provider. Over time, these siloed decisions accumulate into architecture debt that no single team recognizes until it is too late.

popularity masks important differences: One is tactical and correctable, the other is strategic and deeply structural. By calling everything “technical debt,” companies miss the chance to create practices, funding models and remediation plans specifically designed for architecture debt.

Diagnosing Architecture Debt

For specialists, the key is to make the invisible visible. Techniques include the following.

- **Duplicate platforms:** Count how many systems perform overlapping functions (five human resources systems, three customer-relationship management tools). High duplication signals structural inefficiency.
- **Integration complexity:** Measure the number of point-to-point connections versus API gateways, enterprise service buses (ESBs) or event-driven models.
- **Principle violations:** Track how many systems lack defined owners, documented interfaces or compliance with enterprise standards.
- **Latency chains:** Calculate end-to-end data flow time across multiple hops. If data must pass through six legacy systems before reaching an AI model, architecture debt is the bottleneck.
- **Configuration management database (CMDB) completeness:** Use **ServiceNow** or a similar tool to measure the percentage of applications with filled ownership, life cycle and dependency fields.

These metrics help shift architecture debt from abstract concept to quantifiable problem.

Real-World Cases

AI adoption blocked by data silos

A company builds advanced machine learning (ML) models to forecast demand. The data scientists are skilled, the models promising. But data resides in five separate legacy systems with no unified schema. Integration projects drag on for months. The result: The AI program stalls, not because of poor algorithms, but because of architecture debt in data pipelines.

Cloud migration and the immovable legacy

During a cloud migration, 30% of applications can't move. They depend on outdated software, proprietary protocols or undocumented dependencies. The migration

Resilience gaps in infrastructure

An enterprise invests heavily in monitoring and incident response. Still, outages persist. The real culprit: an outdated network architecture designed for 2005 traffic patterns, not modern workloads. This is architecture debt in infrastructure design, invisible to dashboards focused only on uptime.

The Consequences of Ignoring Architecture Debt

- **Financial waste:** Garner and McKinsey studies show that up to 40% of digital transformation budgets are consumed by untangling hidden architectural problems. Because these costs rarely appear in business cases, programs that start with a \$50 million budget can balloon to \$70 million to \$80 million once hidden dependencies surface.
- **Program delays:** Architectural weaknesses extend timelines dramatically. A 12-month program often **drifts into 24 to 36 months** once hidden silos or outdated middleware emerge, and many deliver only a fraction of their intended value because resources are consumed by firefighting. For specialists, unresolved architecture debt is the single biggest threat to predictable delivery – more damaging than resourcing gaps or shifting requirements.
- **Technological stagnation:** Architecture debt locks companies into the past. AI, automation and cloud adoption initiatives don't fail because teams lack skill. They fail because underlying systems are fragmented, unscalable or incompatible. In fact, 70% to 80% of AI projects **fail to scale** beyond pilot. Without a unified data platform, AI models never leave the lab. Without rationalized applications, automation pilots don't scale. Without decoupled integration, cloud migrations stall. The result: Organizations spend years talking about innovation while still running critical processes on legacy mainframes or fragile middleware.
- **Increased risk exposure:** Old integrations and outdated platforms are not just inefficient – they are dangerous. Legacy systems were a contributing factor in **over 60% of major cyber incidents**. Architecture debt creates hidden single points of failure, undocumented dependencies and weak security perimeters. A minor outage in one system can cascade into a businesswide incident. Similarly, fragmented landscapes increase the attack surface for cyber threats, while lack of **resilience planning undermines business continuity**. For highly regulated industries, this also translates into compliance risk: Auditors increasingly expect evidence that architectural controls are in place, not just operational ones.
- **Erosion of trust (*new, expanded consequence*):** Perhaps the most subtle consequence is the erosion of trust between IT and the business. When every initiative runs over budget, misses deadlines or delivers less than promised, stakeholders lose faith in IT's ability to execute. This reputational damage is difficult to repair and often leads to shadow IT – business units bypassing



What Can Be Done: 5 Steps

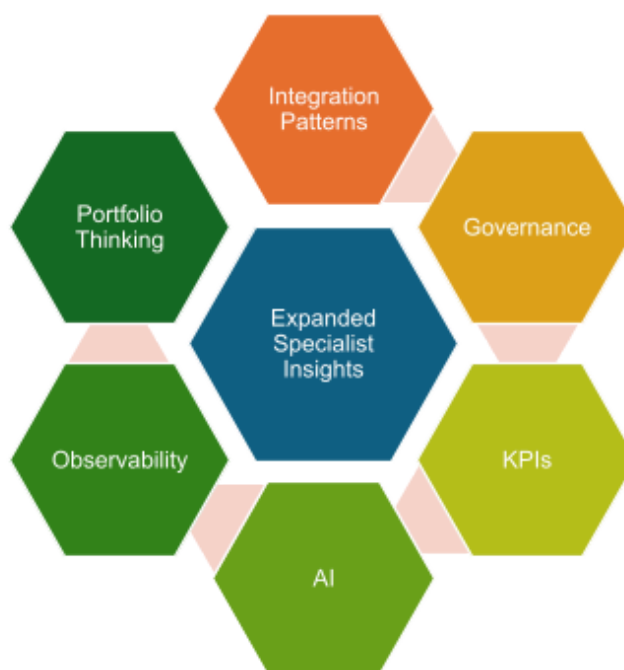
1. **Officially define architecture debt.** Organizations must recognize architecture debt as a category distinct from technical debt. This requires clear definitions and communication to both technical and business leaders.
2. **Build metrics and dashboards.** Track the following:
 - The percentage of systems without owners.
 - Point-to-point integrations.
 - CMDB completeness and accuracy.
 - The percentage of platforms aligned with architectural principles.
3. **Practice architecture observability.** Just as site reliability engineers monitor service reliability, architects must monitor the structural health of the IT landscape: scalability, modularity, principle adherence.
4. **Run architecture reviews.** Beyond code reviews, organizations should hold systematic architecture reviews for projects and domains.
5. **Manage debt as a portfolio.** Not all debt needs immediate repayment. Like managing a financial portfolio, organizations should prioritize by business impact — addressing the costliest blockers first while consciously tolerating manageable inefficiencies.

Expanded Specialist Insights

- **Integration patterns matter.** The choice of integration architecture is often the difference between manageable complexity and long-term paralysis. Point-to-point integrations may work in the short term, but they scale poorly. Every new system exponentially increases dependencies. By contrast, API-first strategies and event-driven architectures decouple services, reduce **systemic coupling and improve resilience**. For specialists, this isn't just a design preference, it's a debt-management strategy. Shifting from legacy ESB spaghetti to asynchronous event hubs or API gateways can lower failure domains, speed up onboarding of new services and cut integration costs by double digits.

can be repaid. Effective governance creates a feedback loop: Each project is reviewed not just for delivery but for alignment with long-term enterprise principles. Specialists know that without governance, every “local optimization” becomes tomorrow’s systemic problem. For example, allowing teams to adopt overlapping SaaS tools without architectural oversight results in fragmented landscapes that later block consolidation and automation.

- **Link debt to business KPIs.** Architecture debt becomes meaningful to executives only when it connects to business outcomes. Instead of abstract warnings, architects should show how debt directly slows time to market, increases operating cost per transaction or reduces system resilience during incidents. For example, duplicating finance platforms may not look critical until leaders see the effect: increased reconciliation time, higher audit risks and slower financial close — a challenge reported by 42% of CFOs in [Deloitte’s 2023 Finance Transformation Survey](#). By translating architectural debt into the same KPIs tracked at the board level, specialists ensure visibility and funding for remediation.



⊕ Zoom

- **Architecture debt and AI.** The rise of AI amplifies the risks of architecture debt. AI models thrive on clean, accessible and governed data, yet architecture debt often means data silos, inconsistent schemas and poor lineage tracking. Even world-class models will fail to scale if pipelines are fragmented or unobservable. For specialists, this means architecture is no longer just an IT foundation, it is a direct enabler of AI success. A single broken data lineage can undermine model explainability, making regulatory compliance impossible. Conversely, organizations that actively reduce architecture debt in their data ecosystems gain a competitive edge: Their AI initiatives reach production faster and deliver measurable business outcomes.

means tracking system dependencies, integration bottlenecks and principal compliance in near-real time. For example, dashboards that show the percentage of undocumented APIs or latency across data pipelines allow architects to detect architecture debt before it cripples delivery. Just as site reliability engineers measure reliability, architects must measure structural health.

- **Adopt portfolio thinking for debt remediation.** Not all debt can or should be repaid immediately. Treating debt as a portfolio creates discipline: Some items are high-interest and must be resolved quickly (such as undocumented critical integrations), while others can be tolerated if managed (two overlapping SaaS tools in low-risk domains). Using risk-adjusted prioritization allows architects to align remediation with business value, rather than chasing every inefficiency. This approach also helps secure executive funding by linking remediation plans to measurable return on investment.

Conclusion

Technical debt is visible. Everyone can point to buggy code, missing tests or a legacy function that needs rewriting. Architecture debt is hidden, and that makes it far more dangerous. It accumulates quietly in duplicated platforms, fragile integrations and outdated governance models. And while technical debt slows delivery, architecture debt stalls entire transformations.

Looking ahead, organizations that fail to address architecture debt will struggle to adopt AI at scale, modernize for cloud or meet rising cybersecurity and compliance demands. The winners will be those that treat architecture debt as a board-level risk and invest in continuous architecture observability, governance and remediation.

For specialists, the message is clear: Stop treating “tech debt” as a catch-all phrase. Build the practices, metrics and governance to make architecture debt visible and actionable.

In the era of AI and data-driven enterprises, reducing architecture debt will no longer be a technical choice. It will be a strategic differentiator that separates the companies that can transform from those that will fall behind.



future of enterprise architecture and digital transformation. Nadzeya is an active member of leading industry organizations, serving...

[Read more from Nadzeya Stalbouskaya →](#)

TNS owner Insight Partners is an investor in: Real.

SHARE THIS STORY



TRENDING STORIES

1. Groundcover eyes visibility gap in agentic AI monitoring by targeting multi-step workflows
2. SRE Housekeeping: Where to Get Started for New Hires
3. Andrej Karpathy's 630-line Python script ran 50 experiments overnight without any human input
4. Technical Debt vs. Architecture Debt: Don't Confuse Them
5. Kubernetes co-founder Brendan Burns: AI-generated code will become as invisible as assembly

INSIGHTS FROM OUR SPONSORS

Announcing Google BigQuery Target

25 August 2026

Iceberg CDC Merge on Read for Every Query Engine

28 June 2026

Announcing MotherDuck and DuckDB Targets

17 June 2026



chainguard

This Shit is Hard: Getting commercial software into regulated environments using AI

22 September 2026

The flood is coming and the pipes were already full

15 September 2026

Announcing the Sovereign Artifacts beta

15 September 2026



HEROKU
from Salesforce

6 Lesser-Known Heroku CLI Commands You Probably Aren't Using

25 August 2026

Fine-Grained Access Control Now Available for All Heroku Customers

21 August 2026

How To Set Up a Staging Environment on Heroku in 3 Easy Steps

16 July 2026

most recent TNS articles in your inbox each day.

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)[Sponsors](#)[Advertise With Us](#)[Contributions](#)

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)[Backend Developer Roadmap](#)[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)[Terms of Use](#)[Advertising Terms & Conditions](#)[Privacy Policy](#)[Cookie Policy](#)